

LocalActionDia- grams

**Super awesome package for local action
diagrams.**

0.1

29 January 2023

Marcus Chijoff
Stephan Tornier

Marcus Chijoff Email: marcus.chijoff@uon.edu.au

Homepage: <https://newcastle.edu.au>

Address: No

Stephan Tornier Email: stephan.tornier@newcastle.edu.au

Homepage: <https://newcastle.edu.au>

Address: No

Contents

1	RSGraphs	3
1.1	Creating RSGraphs	3
1.2	RSGraph Attributes and Properties	5
1.3	RSGraph Operations	8
2	Local Action Diagrams	11
2.1	Creating Local Action Diagrams	11
2.2	Local Action Diagram Attributes and Properties	13
3	IO Operations and Visualisation	20
3.1	IO Operations	20
4	Isomorphisms and Automorphisms	24
5	Enumeration	25
	Index	26

Chapter 1

RSGraphs

An RSGraph is a graph $\Gamma = (V, A, o, r)$ that consists of a vertex set V , an arc set A , a map $o : A \rightarrow V$ which assigns each vertex to an *origin*, and a bijection $r : A \rightarrow A$ which assigns each arc to a *reverse* arc. There is also a map $t := o \circ r$ which assigns each arc to a *terminus* vertex. Note the RSGraphs are allowed to contain loops and parallel arcs.

We created the RSGraph object for this package due to our use of graphs having a specialised definition. For example, every arc in an RSGraph must have a reverse arc associated with it while a *Digraph* from the *Digraphs* allows for arcs that do not have an associated reverse arc. Creating a new object category for RSGraphs allows for both better error checking when using RSGraphs and to avoid conflicting definitions, such as the definition of a cycle for a *Digraph* and an RSGraph. We provide functionality to convert between an RSGraph and a *Graph* from the *GRAPE* package or a *Digraph* from the *Digraphs* package.

The name "RSGraph" arises from Reid and Smith, the authors of the local action diagram paper. This name was chosen to avoid conflicting with the *GRAPE* package.

1.1 Creating RSGraphs

1.1.1 IsRSGraph (for an object)

▷ `IsRSGraph(obj)` (Category)

Returns: `true` if *graph* is of the category `IsRSGraph` and `false` otherwise.

Every RSGraph object belongs to the category `IsRSGraph`. Furthermore, every RSGraph is an immutable attribute storing object.

1.1.2 RSGraphByAdjacencyList (for a list, permutation[, and list])

▷ `RSGraphByAdjacencyList(adjacency_list, reverse_map[, vertex_ids])` (operation)

▷ `RSGraphByAdjacencyListNC(adjacency_list, reverse_map[, vertex_ids])` (operation)

Returns: An RSGraph.

This function creates an RSGraph described by an adjacency list. The argument *adjacnecy_list* is a list of the form `[[origin, terminus], ...]`. Each arc in the graph is given an id which corresponds to its position in the list *adjacency_list*.

The reverse map must be a permutation that is an involution --- i.e. `reverse_map*reverse_map = ()`. Furthermore, if element *idx* of *adjacency_list* is `[u, v]` then *adjacency_list*`[idx~reverse_map]` must equal `[v, u]`.

By default, the vertices are labelled with ids $\{1, 2, \dots, n\}$ where n is the maximal element of `Flat(adjacency_list)`. If the optional third argument `vertex_ids` is given then these are taken to be the vertex ids. This argument must be a dense list of integers.

Finally, the underlying graph must be connected.

The NC variant of the function does not check that the graph is connected, the reverse map is valid, and that each arc has an associated reverse arc.

Example

```
gap> adj_list := [[1, 2], [2, 1], [2, 2], [2, 2], [1, 1]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);
<RSGraph with 2 vertices and 5 arcs>

gap> adj_list2 := [[2, 2]];;
gap> rev_map2 := ();;
gap> vertex_ids := [2];;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map, vertex_ids);
<RSGraph with 1 vertex and 1 arc>
```

1.1.3 RSGraphByAdjacencyMatrix (for a matrix, permutation[, and list])

▷ `RSGraphByAdjacencyMatrix(adjacency_matrix, reverse_map[, vertex_ids])` (operation)
 ▷ `RSGraphByAdjacencyMatrixNC(adjacency_matrix, reverse_map[, vertex_ids])` (operation)

Returns: An `RSGraph`.

This function creates an `RSGraph` described by an adjacency matrix. The argument `adjacency_matrix` is an $n \times n$ matrix represented by a list of lists. This means that there are m arcs between vertices u and v of the graph if `adjacency_matrix[u][v] = m`.

The matrix is read in row-major order to determine the arc ids of the graph. This is important for determining the reverse map of the graph. The reverse map must be a permutation that is an involution --- i.e. `reverse_map*reverse_map = ()`.

By default, the vertices are labelled with ids $\{1, 2, \dots, n\}$ where $n = \text{Size}(\text{adjacency_matrix})$ --- i.e. the number of rows (and columns) of the matrix. If the optional third argument `vertex_ids` is given then these are taken to be the vertex ids. This argument must be a dense list of integers and element `idx` of the list will be the label of the vertex in row and column `idx` of `adjacency_matrix`.

Finally, the underlying graph must be connected.

Note that the first argument must be a list of lists with each sublist containing integers. In particular, it can not be an object in the category `IsMatrixObj` constructed by the function `Matrix`.

The NC variant of the function does not check that the graph is connected, the reverse map is valid, and that each arc has an associated reverse arc.

Example

```
gap> adj_mat := [[1, 2], [2, 0]];;
gap> rev_map := (2, 4)(3,5);;
gap> vertex_ids := [2,3];;
gap> graph := RSGraphByAdjacencyMatrix(adj_list, rev_map, vertex_ids);
<RSGraph with 2 vertices and 5 arcs>

gap> Print(graph);
Vertices = { 2, 3 }
Arcs = {
```

```

1 = ( origin = 2, terminus = 2, inverse = 1 )
2 = ( origin = 2, terminus = 3, inverse = 4 )
3 = ( origin = 2, terminus = 3, inverse = 5 )
4 = ( origin = 3, terminus = 2, inverse = 2 )
5 = ( origin = 3, terminus = 2, inverse = 3 )
}
Reverse Map = (2,4)(3,5)

```

1.2 RSGraph Attributes and Properties

1.2.1 RSGraphVertices

- ▷ `RSGraphVertices(graph)` (attribute)
Returns: The list of vertex ids of *graph*.

1.2.2 RSGraphNumberVertices

- ▷ `RSGraphNumberVertices(graph)` (attribute)
Returns: The number of vertices of *graph*.

1.2.3 RSGraphArcs

- ▷ `RSGraphArcs(graph)` (attribute)
Returns: The record of arcs of *graph*.

The name of each component of the record is the id of the arc. Each component is itself a record with three components: origin, terminus, and inverse. These store the origin vertex, terminus vertex, and inverse arc id respectively.

For iterating over the arcs of an `RSGraph` see `RSGraphArcIterator` (1.3.1).

Example

```

gap> adj_list := [[1, 2], [2, 1], [2, 2], [2, 2], [1, 1]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);
<RSGraph with 2 vertices and 5 arcs>
gap> RSGraphArcs(graph).1;
rec( inverse := 2, origin := 1, terminus := 2 )

```

1.2.4 RSGraphNumberArcs

- ▷ `RSGraphNumberArcs(graph)` (attribute)
Returns: The number of arcs of *graph*.

1.2.5 RSGraphArcIDs

- ▷ `RSGraphArcIDs(graph)` (attribute)
Returns: The list of arc ids of *graph*.

1.2.6 RSGraphReverseMap

- ▷ RSGraphReverseMap(*graph*) (attribute)
Returns: The reverse map of *graph*.

1.2.7 RSGraphAdjacencyMatrix

- ▷ RSGraphAdjacencyMatrix(*graph*) (attribute)
Returns: The adjacency matrix of *graph*.

1.2.8 RSGraphOutNeighbours

- ▷ RSGraphOutNeighbours(*graph*) (attribute)
Returns: The record of vertex neighbours of *graph*.

The name of each component is a vertex id of the graph. Each element is a list of vertex ids corresponding to the neighbours of the vertex. A vertex id *v_id* is in list *idx* if and only if there is an arc connecting the vertices *idx* and *v_id*. For example, if there is an arc from vertex 1 to 3 of the graph then 3 is an element of RSGraphOutNeighbours(*graph*).1.

1.2.9 RSGraphInNeighbours

- ▷ RSGraphInNeighbours(*graph*) (attribute)
Returns: The record of vertex neighbours of *graph*.

Since every arc has an inverse arc this returns the same result as RSGraphOutNeighbours(*graph*).

1.2.10 RSGraphOutArcs

- ▷ RSGraphOutArcs(*graph*) (attribute)
Returns: The record of arcs originating at each vertex.

The name of each component is a vertex id of the graph. Each element is a list of arc ids. An arc id is in list *idx* if and only if the origin of that arc is *idx*. For example, if an arc with id 2 originates at vertex 3 then 2 is an element of RSGraphInNeighbours(*graph*).3.

1.2.11 RSGraphInArcs

- ▷ RSGraphInArcs(*graph*) (attribute)
Returns: The record of arcs originating at each vertex.

The name of each component is a vertex id of the graph. Each element is a list of arc ids. An arc id is in list *idx* if and only if the terminus of that arc is *idx*. For example, if an arc with id 2 terminates at vertex 1 then 2 is an element of RSGraphInNeighbours(*graph*).1.

Example

```
gap> adj_list := [[1, 2], [2, 1], [2, 2], [2, 2]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);
<RSGraph with 2 vertices and 4 arcs>
gap> RSGraphOutNeighbours(graph);
rec( 1 := [ 2 ], 2 := [ 1, 2 ] )
gap> RSGraphInNeighbours(graph);
```

```

rec( 1 := [ 2 ], 2 := [ 1, 2 ] )
gap> RSGraphOutArcs(graph);
rec( 1 := [ 1 ], 2 := [ 2, 3, 4 ] )
gap> RSGraphInArcs(graph);
rec( 1 := [ 2 ], 2 := [ 1, 3, 4 ] )

```

1.2.12 RSGraphBipartition

▷ RSGraphBipartition(*graph*) (attribute)

Returns: The bipartition of *graph* if it is bipartite and fail otherwise.

If the *graph* is bipartite then this returns a list containing two lists. These two lists contain the vertex ids in each bipartition.

Note that for an RSGraph to be bipartite it must not contain parallel edges or loops.

Example

```

gap> adj_list := [[1, 2], [2, 1], [1, 3], [3, 1]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);
<RSGraph with 3 vertices and 4 arcs>
gap> RSGraphBipartition(graph);
[ [ 1 ], [ 2, 3 ] ]

gap> adj_list := [[1, 2], [2, 1], [2, 2], [2, 2]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);
<RSGraph with 2 vertices and 4 arcs>
gap> RSGraphBipartition(graph);
fail

```

1.2.13 RSGraphDegree

▷ RSGraphDegree(*graph*) (attribute)

Returns: The maximum number of arcs originating/terminating at any vertex in the *graph*.

1.2.14 RSGraphIsCycle

▷ RSGraphIsCycle(*graph*) (property)

Returns: true if *graph* is a cycle graph and false otherwise.

This is a cycle in the sense of [cite RS paper here](#)!!!. A graph is a cycle if:

- It has a single vertex, two arcs, and a non-trivial reverse map.
- It has two vertices and two edges (i.e. four arcs) between them.
- It has three or more vertices, no loops, and all vertices have degree two.

1.2.15 RSGraphIsBipartite

▷ RSGraphIsBipartite(*graph*) (property)

Returns: true if *graph* is a bipartite graph and false otherwise.

This is equivalent to RSGraphBipartition(*graph*) <> fail.

1.2.16 RSGraphHasParallelArcs

▷ `RSGraphHasParallelArcs(graph)` (property)

Returns: true if *graph* has multiple arcs in the same direction between any two pairs of vertices.

1.3 RSGraph Operations

1.3.1 RSGraphArcIterator

▷ `RSGraphArcIterator(graph)` (operation)

Returns: An iterator which iterates over the arcs of *graph*.

This functions returns an iterator object which iterates over the arcs of *graph*. Each element of the iterator is of the form `[arc_id, arc_record]` where *arc_record* is in the form described in `RSGraphArcs` (1.2.3).

Example

```
gap> adj_list := [[1, 2], [2, 1], [2, 2]];;
gap> rev_map := (1,2);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);;
gap> for arc in RSGraphArcIterator(graph) do
gap>   View(arc);
gap>   Print("\n");
gap> od;
[ 1, rec( inverse := 2, origin := 1, terminus := 2 ) ]
[ 2, rec( inverse := 1, origin := 2, terminus := 1 ) ]
[ 3, rec( inverse := 3, origin := 2, terminus := 2 ) ]
```

1.3.2 RSGraphSubgraph (for an RSGraph and List of arcs)

▷ `RSGraphSubgraph(graph, arc_list)` (operation)

▷ `RSGraphSubgraphNC(graph, arc_list)` (operation)

Returns: An arc induced subgraph of *graph*.

Given a graph and list of arcs in this graph, this function returns the subgraph consisting exactly of those arcs and the vertices connecting them. The vertex ids, arc ids, and reverse map are preserved. Note that the reverse map is not changed and so may act on a larger set of integers than is strictly necessary.

If an arc is in *arc_list* then its reverse must also be in it. Furthermore, the resulting subgraph must be connected.

The NC variant of this function does not check for connectivity or for the inclusion of arc reversals.

Example

```
gap> adj_list := [[1, 2], [2, 1], [1, 2], [2, 1], [2, 2]];;
gap> rev_map := (1,2)(3,4);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);;
gap> Print(graph);
Vertices = { 1, 2 }
Arcs = {
  1 = ( origin = 1, terminus = 2, inverse = 2 )
  2 = ( origin = 2, terminus = 1, inverse = 1 )
  3 = ( origin = 1, terminus = 2, inverse = 4 )
```

```

    4 = ( origin = 2, terminus = 1, inverse = 3 )
    5 = ( origin = 2, terminus = 2, inverse = 5 )
  }
Reverse Map = (1,2)(3,4)
gap> subgraph := RSGraphSubgraph(graph, [1,2,5]);
Print(subgraph);
Vertices = { 1, 2 }
Arcs = {
    1 = ( origin = 1, terminus = 2, inverse = 2 )
    2 = ( origin = 2, terminus = 1, inverse = 1 )
    5 = ( origin = 2, terminus = 2, inverse = 5 )
}
Reverse Map = (1,2)(3,4)

```

1.3.3 RSGraphToStandardForm

▷ `RSGraphToStandardForm(graph)` (operation)

Returns: A record containing the graph in a standard range and the maps from the old graph to the new graph.

Given an arbitrary RSGraph with N vertices and M arcs this function changes the vertex ids to be in $\{1, 2, \dots, N\}$ and arc ids to be in the range $\{1, 2, \dots, M\}$. It also ensures that the arcs are sorted in lexicographical order with respect to the origin and terminus vertices.

If Γ is the *graph* then the vertex mapping is defined by $V(\Gamma)_i \mapsto i$. The arcs origin and terminus vertices then have this mapping applied to them. The resulting arcs are then sorted in lexicographical order so that an arc with a smaller arc id will have an origin vertex with a smaller id than one with a larger arc id. If the origin vertices are equal then the arc with the smaller terminus id will have a smaller arc id. The reverse map of the graph is changed to have the same action on the new arc ids.

This function returns a record containing three components. The *graph* component contains the new graph constructed from it, the *vertex_id_map* component contains the map from the vertex ids of the original graph to the new graph, and the *arc_id_map* component contains the map from the arc ids of the original graph to the new graph.

The *vertex_id_map* and *arc_id_map* are both in the category `IsConstantTimeAccessGeneralMapping`.

Example

```

gap> adj_list := [[2, 3], [3, 2], [2, 3], [3, 2], [3, 3]];
gap> rev_map := (1,2)(3,4);;
gap> vertex_ids := [2, 3];;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map, vertex_ids);;
gap> Print(graph);
Vertices = { 2, 3 }
Arcs = {
    1 = ( origin = 2, terminus = 3, inverse = 2 )
    2 = ( origin = 3, terminus = 2, inverse = 1 )
    3 = ( origin = 2, terminus = 3, inverse = 4 )
    4 = ( origin = 3, terminus = 2, inverse = 3 )
    5 = ( origin = 3, terminus = 2, inverse = 5 )
}
Reverse Map = (1,2)(3,4)
gap> standard_rec := RSGraphToStandardForm(graph);;
gap> Print(standard_rec.graph);

```

```

Vertices = { 1, 2 }
Arcs = {
    1 = ( origin = 1, terminus = 2, inverse = 3 )
    2 = ( origin = 1, terminus = 2, inverse = 4 )
    3 = ( origin = 2, terminus = 1, inverse = 1 )
    4 = ( origin = 2, terminus = 1, inverse = 2 )
    5 = ( origin = 2, terminus = 2, inverse = 5 )
}
Reverse Map = (1,3)(2,4)
gap> Print(2~standard_rec.arc_id_map);
3

```

1.3.4 RSGraphSpanningTree

▷ RSGraphSpanningTree(*graph* [, *search_type*])

(operation)

Returns: A spanning tree of the graph.

This function calculates a spanning tree of the graph based on the edges. This means it does not include and loops or cycles (see RSGraphIsCycle (1.2.14) for the definition of a cycle in an RSGraph). By default it uses a breadth first search.

If the optional parameter *search_type* is equal to the string "dfs" then it will use a depth first search instead. If it is equal to the string "bfs" then it will use a breadth first search. If it is equal to any other string then a warning will be raised and the function will use a breadth first search.

Example

```

gap> adj_list := [[1, 2], [2, 1], [1, 3], [3, 1], [1, 4], [4, 1], \
gap>               [2, 3], [3, 2], [3, 4], [4, 3]];
gap> rev_map := (1,2)(3,4)(5,6)(7,8)(9,10);;
gap> graph := RSGraphByAdjacencyList(adj_list, rev_map);;
gap> tree_bfs := RSGraphSpanningTree(graph);;
gap> Print(tree_bfs);
Vertices = { 1, 2, 3, 4 }
Arcs = {
    1 = ( origin = 1, terminus = 2, inverse = 2 )
    2 = ( origin = 2, terminus = 1, inverse = 1 )
    3 = ( origin = 1, terminus = 3, inverse = 4 )
    4 = ( origin = 3, terminus = 1, inverse = 3 )
    5 = ( origin = 1, terminus = 4, inverse = 6 )
    6 = ( origin = 4, terminus = 1, inverse = 5 )
}
Reverse Map = (1,2)(3,4)(5,6)(7,8)(9,10)
gap> tree_dfs := RSGraphSpanningTree(graph, "dfs");;
gap> Print(tree_dfs);
Vertices = { 1, 2, 3, 4 }
Arcs = {
    1 = ( origin = 1, terminus = 2, inverse = 2 )
    2 = ( origin = 2, terminus = 1, inverse = 1 )
    7 = ( origin = 2, terminus = 3, inverse = 8 )
    8 = ( origin = 3, terminus = 2, inverse = 7 )
    9 = ( origin = 3, terminus = 4, inverse = 10 )
    10 = ( origin = 4, terminus = 3, inverse = 9 )
}
Reverse Map = (1,2)(3,4)(5,6)(7,8)(9,10)

```

Chapter 2

Local Action Diagrams

A local action diagram $\Delta = (\Gamma, (X_a), (G(v)))$ consists of:

- a graph Γ (an `RSGraph`),
- for each arc $a \in A(\Gamma)$ a *colour set* $|X_a|$ disjoint from every other colour set,
- for each vertex $v \in V(\Gamma)$ a group $G(v)$ labelling the vertex such that $G(V)$ acts on $\bigsqcup_{a \in o^{-1}(v)} X_a$ and the orbits of $G(V)$ are precisely the colour sets $|X_a|$ for $a \in o^{-1}(v)$.

Local action diagrams are a powerful tool for studying (P) -closed groups because isomorphism classes of local action diagrams are in a one-to-one correspondence with conjugacy classes of (P) -closed groups. Furthermore, properties of the corresponding (P) -closed group can be determined from the local action diagram. In the case when a local action diagram is finite, we can represent the diagram in `GAP` and determine properties of the (potentially infinite) corresponding group.

2.1 Creating Local Action Diagrams

2.1.1 `IsLocalActionDiagram` (for an object)

▷ `IsLocalActionDiagram(obj)` (Category)

Returns: true if the object is of the category `IsLocalActionDiagram` and false otherwise.

Every local action diagram belongs to the category `IsLocalActionDiagram`. Furthermore, every local action diagram is an immutable attribute storing object.

2.1.2 `LocalActionDiagramFromData` (for an `RSGraph`, list or record of vertex labels, and list or record of arc labels)

▷ `LocalActionDiagramFromData(graph, vertex_labels, arc_labels)` (operation)

▷ `LocalActionDiagramFromDataNC(graph, vertex_labels, arc_labels)` (operation)

Returns: A local action diagram object.

This function constructs a local action diagram by having all of the diagrams data given to it. This is done by providing the `RSGraph` of the diagram and either lists of vertex and arc labels or records of vertex and arc labels.

If two lists are provided then the vertex and arc labels are assumed to be in the same order as `RSGraphVertices(graph)` and `RSGraphArcIDs(graph)` respectively. This

means, for example, that element `idx` of `vertex_labels` is the label of the vertex with id `RSGraphVertexIDs(graph)[idx]`. If two records are provided then the names of the record elements are the vertex and arc ids respectively and the components are their labels.

The vertex labels are permutation groups. The domain each label acts on is stored in the mutable attribute `PermGroupDomain` (see `PermGroupDomain(??)`). By default, this is equal to the `MovedPoints` of the group but can be changed to allow for the group to have fixed points.

The arc labels are lists of integers. Each arc must be labelled by by an orbit of the vertex label at the vertex that arc originates at. Furthermore, every vertex must have an arc originating at it for each orbit of its vertex label. Note that the orbits of a vertex label G are determined by `Orbits(G, PermGroupDomain(G))`.

Unlike the formal definition of a local action diagram, we allow for overlap between the domains of different vertex labels. For example, a two vertex local action diagram could be labelled by C_2 and S_3 and they can have domains $\{1, 2\}$ and $\{1, 2, 3\}$ respectively.

The NC variant of the function does not check that there is a label for each vertex and arc (and exactly this many labels), that the domains of the groups match up with the arc labels, or that the orbits of the vertex labels match up with the arc labels.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1], [1, 2], [2, 1]], (2,3));;
gap> label_1 := Group((1,2));;
gap> SetPermGroupDomain(label_1, [1, 2, 3]);;
gap> label_2 := Group((1,2,3));;
gap> vertex_labels := [label_1, label_2];
gap> arc_labels := [[1, 2], [3], [1,2,3]];
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);
<LocalActionDiagram with 2 vertices and 3 arcs>
gap> Print(lad);
Vertices = { 1, 2 }
Arcs = {
    1 = ( origin = 1, terminus = 1, inverse = 1 )
    2 = ( origin = 1, terminus = 2, inverse = 3 )
    3 = ( origin = 2, terminus = 1, inverse = 2 )
}
Reverse Map = (2,3)
Vertex Labels = {
    1 = Group( [ (1,2) ] )
    2 = Group( [ (1,2,3) ] )
}
Arc Labels = {
    1 = [ 1, 2 ]
    2 = [ 3 ]
    3 = [ 1, 2, 3 ]
}
```

2.1.3 LocalActionDiagramFromBurgerMozesUniversalGroup

▷ `LocalActionDiagramFromBurgerMozesUniversalGroup(perm_group)`

(operation)

Returns: A local action diagram object.

Given a permutation group F (`perm_group`) this function returns the local action diagram corresponding to the Burger–Mozes universal group $U(F)$ (see [cite Burger–Mozes and Colin and Simon

for construction?])). This is a local action diagram with one vertex labelled by F and a self-reverse loop for each orbit of F . Note that F acts on the domain $\text{PermGroupDomain}(\text{perm_group})$.

Example

```
gap> lad := LocalActionDiagramFromBurgerMozesUniversalGroup(Group((1,2)(3,4)));
<U(Group( [ (1,2)(3,4) ] )) (as a Local Action Diagram)>
```

2.1.4 LocalActionDiagramConstructBipartitionPreserving

▷ `LocalActionDiagramConstructBipartitionPreserving(lad)` (operation)

Returns: A local action diagram object.

Given a single vertex local action diagram Δ (lad) corresponding to the group $U(\Delta)$ acting on a tree T this function returns the local action diagram corresponding to the group $U(\Delta)^\circ = \{g \in U(\Delta) \mid \forall v \in V(T), d(v, gv) \equiv 0 \pmod{2}\}$. Intuitively this is the group of all automorphisms the preserve the bipartition of T which always exists since a single vertex local action diagram is vertex transitive.

Given a group G labelling the single vertex of Δ and arc labels X_a r this local action diagram is constructed as follows:

- Make a graph with two vertices and an edge for each loop of the graph for Δ .
- Label both vertices by G .
- For each arc a in the graph of Δ label the corresponding arc in the new graph by X_a and the reverse of this arc by $X_{\bar{a}}$.

2.2 Local Action Diagram Attributes and Properties

2.2.1 LocalActionDiagramRSGraph

▷ `LocalActionDiagramRSGraph(lad)` (attribute)

Returns: RSGraph associated with the local action diagram.

2.2.2 LocalActionDiagramVertexLabels

▷ `LocalActionDiagramVertexLabels(lad)` (attribute)

Returns: Vertex labels of the local action diagram.

2.2.3 LocalActionDiagramArcLabels

▷ `LocalActionDiagramArcLabels(lad)` (attribute)

Returns: Arc labels of the local action diagram.

2.2.4 LocalActionDiagramVertices

▷ `LocalActionDiagramVertices(lad)` (attribute)

Returns: The vertices of the local action diagrams graph.

Shorthand for `RSGraphVertices(LocalActionDiagramRSGraph(lad))`.

2.2.5 LocalActionDiagramArcs

- ▷ `LocalActionDiagramArcs(lad)` (attribute)
Returns: The arcs of the local action diagrams graph.
 Shorthand for `RSGraphArcs(LocalActionDiagramRSGraph(lad))`.

2.2.6 LocalActionDiagramArcIDs

- ▷ `LocalActionDiagramArcIDs(lad)` (attribute)
Returns: The arc ids of the local action diagrams graph.
 Shorthand for `RSGraphArcIDs(LocalActionDiagramRSGraph(lad))`.

2.2.7 LocalActionDiagramNumberVertices

- ▷ `LocalActionDiagramNumberVertices(lad)` (attribute)
Returns: The number of vertices in the local action diagrams graph.
 Shorthand for `RSGraphNumberVertices(LocalActionDiagramRSGraph(lad))`.

2.2.8 LocalActionDiagramNumberArcs

- ▷ `LocalActionDiagramNumberArcs(lad)` (attribute)
Returns: The number of arcs in the local action diagrams graph.
 Shorthand for `RSGraphNumberArcs(LocalActionDiagramRSGraph(lad))`.

2.2.9 LocalActionDiagramReverseMap

- ▷ `LocalActionDiagramReverseMap(lad)` (attribute)
Returns: The reverse map of the local action diagrams graph.
 Shorthand for `RSGraphReverseMap(LocalActionDiagramRSGraph(lad))`.

2.2.10 LocalActionDiagramOutNeighbours

- ▷ `LocalActionDiagramOutNeighbours(lad)` (attribute)
Returns: The record of out neighbours of the local action diagrams graph.
 Shorthand for `RSGraphOutNeighbours(LocalActionDiagramRSGraph(lad))`.

2.2.11 LocalActionDiagramInNeighbours

- ▷ `LocalActionDiagramInNeighbours(lad)` (attribute)
Returns: The record of in neighbours of the local action diagrams graph.
 Shorthand for `RSGraphInNeighbours(LocalActionDiagramRSGraph(lad))`.

2.2.12 LocalActionDiagramOutArcs

- ▷ `LocalActionDiagramOutArcs(lad)` (attribute)
Returns: The record of out arcs of the local action diagrams graph.
 Shorthand for `RSGraphOutArcs(LocalActionDiagramRSGraph(lad))`.

2.2.13 LocalActionDiagramInArcs

▷ `LocalActionDiagramInArcs(lad)` (attribute)

Returns: The record of in arcs of the local action diagrams graph.
Shorthand for `RSGraphInArcs(LocalActionDiagramRSGraph(lad))`.

2.2.14 LocalActionDiagramGroupName

▷ `LocalActionDiagramGroupName(lad)` (attribute)

Returns: A name for the associated universal group of the local action diagram.

This is a mutable attribute which stores a human readable name for the associated universal group. By default it is equal to the empty string. In this case the `View` function will print `<LocalActionDiagram with n vertices and m arcs>` where n is the number of vertices and m is the number of arcs. If this attribute is set to to any other string then the `View` function will print `<{group_name} (as a Local Action Diagram)>`.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1]], ());
gap> vertex_labels := [SymmetricGroup(3)];
gap> arc_labels := [[1,2,3]];
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);
<LocalActionDiagram with 1 vertex and 1 arc>
gap> SetLocalActionDiagramGroupName(lad, "Aut(T_3)");
gap> View(lad);
<Aut(T_3) (as a Local Action Diagram)>
```

2.2.15 LocalActionDiagramRegularTree

▷ `LocalActionDiagramRegularTree(lad)` (attribute)

Returns: The degree of the regular tree the universal group acts on or fail if it does not act on one.

If every vertex label of the local action diagram has a domain of size d then the associated universal group acts on a regular tree of degree d . This functions returns d if this is the case and fail otherwise.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 2], [2, 1]], (1,2));
gap> vertex_labels := [SymmetricGroup(3), Group((1,2,3))];
gap> arc_labels := [[1,2,3], [1,2,3]];
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);
gap> LocalActionDiagramRegularTree(lad);
3
gap> vertex_labels_2 := [SymmetricGroup(3), Group((1,2))];
gap> arc_labels_2 := [[1,2,3], [1,2]];
gap> lad_2 := LocalActionDiagramFromData(graph, vertex_labels_2, arc_labels_2);
gap> LocalActionDiagramRegularTree(lad_2);
fail
```

2.2.16 LocalActionDiagramScopos

▷ `LocalActionDiagramScopos(lad)` (attribute)

Returns: The list of scopos of the local action diagram.

(CITE REID SMITH HERE) Let $\Delta = (\Gamma, (X_a), (G(v)))$ be a local action diagram. A strongly confluent partial orientation (*scopo*) of Δ is a subset O of $A(\Gamma)$ such that:

- if $a \in O$ then $\bar{a} \notin O$,
- if $a \in O$ then $|X_a| = 1$, and
- for all $v \in V(\Gamma)$ if O contains an arc a originating at v then O contains all arcs that terminate at v except for $\bar{a}$.

This function performs an iterative search for all scopos of a local action diagram. It stores each scopo as a list of arc ids in the scopo. Note that the empty set is a scopo and this is represented as the empty list.

Example

```
gap> graph := RSGraphByAdjacencyList ([[1, 2], [2, 1]], (1,2));;
gap> group := Group();;
gap> SetPermGroupDomain(group, [1]);;
gap> vertex_labels := [group, group];;
gap> arc_labels := [[1], [1]];
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramScopos(lad);
[ [ ], [ 1 ], [ 2 ] ]
```

2.2.17 LocalActionDiagramGroupType

▷ `LocalActionDiagramGroupType(lad)` (attribute)

Returns: The type of the corresponding universal group.

(CITE REID SMITH HERE) Every group acting on a tree can be split into one of six mutually exclusive types: *fixed vertex*, *edge inversion*, *lineal*, *horocyclic*, *focal*, and *general*. These types can be recognised from the groups corresponding local action diagram by analysing the scopos of the local action diagram (see CITE REID SMITH).

This function first calculates the scopos of the local action diagram (see `LocalActionDiagramScopos` (2.2.16)) and then uses this to determine the type of the corresponding group. It outputs this types as a string.

Example

```
gap> graph := RSGraphByAdjacencyList ([[1, 2], [2, 1]], (1,2));;
gap> group := Group();;
gap> SetPermGroupDomain(group, [1]);;
gap> vertex_labels := [group, group];;
gap> arc_labels := [[1], [1]];
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramGroupType(lad);
"General"
```

2.2.18 LocalActionDiagramIsDiscrete

▷ `LocalActionDiagramIsDiscrete(lad)` (property)

Returns: true if the corresponding universal group is discrete and false otherwise.

This function implements (CITE THEOREM HERE) to determine if the corresponding universal group is discrete. This requires calculation of the diagrams group type (see

`LocalActionDiagramGroupType` (2.2.17)). Furthermore, if the group is discrete then it is also uniscalar and unimodular and so if this function returns `true` then it also sets `LocalActionDiagramIsUniscalar` (2.2.19) and `LocalActionDiagramIsUnimodular` (2.2.20) to `true`.

Example

```
gap> graph := RSGraphByAdjacencyList ([[1, 2], [2, 1]], (1,2));;
gap> group := Group();;
gap> SetPermGroupDomain(group, [1]);;
gap> vertex_labels := [group, group];;
gap> arc_labels := [[1], [1]];
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramIsDiscrete(lad);
true
```

2.2.19 LocalActionDiagramIsUniscalar

▷ `LocalActionDiagramIsUniscalar(lad)` (property)

Returns: `true` if the corresponding universal group is uniscalar and `false` otherwise.

This function implements (CITE THEOREM HERE) to determine if the corresponding universal group is uniscalar. This requires calculation of the diagrams group type (see `LocalActionDiagramGroupType` (2.2.17)). Furthermore, if the group is uniscalar then it is also unimodular so if this function returns `true` then it also sets `LocalActionDiagramIsUnimodular` (2.2.20) to `true`.

Example

```
gap> graph := RSGraphByAdjacencyList ([[1, 2], [2, 1], [2, 2]], (1,2));;
gap> group := Group();;
gap> group_2 := Group((1, 2), (1,2,3), (4,5));;
gap> SetPermGroupDomain(group, [1]);;
gap> vertex_labels := [group, group_2];;
gap> arc_labels := [[1], [1, 2, 3], [4, 5]];
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramIsDiscrete(lad);
false
gap> LocalActionDiagramIsUniscalar(lad);
true
```

2.2.20 LocalActionDiagramIsUnimodular

▷ `LocalActionDiagramIsUnimodular(lad)` (property)

Returns: `true` if the corresponding universal group is unimodular and `false` otherwise.

This function implements (CITE THEOREM HERE both us and BK) to determine if the corresponding universal group is unimodular. This requires calculation of a spanning tree of the graph (see `RSGraphSpanningTree` (1.3.4)).

Example

```
gap> adj_list := [[1, 2], [2, 1], [2, 3], [3, 2], [3, 1], [1, 3], [1, 1]];
gap> graph := RSGraphByAdjacencyList(adj_list, (1,2)(3,4)(5,6));;
gap> vertex_labels := [];;
gap> Add(vertex_labels, Group((1, 2, 3), (4,5), (6,7)));;
gap> Add(vertex_labels, Group((1, 2), (3, 4, 5)));;
```

```

gap> Add(vertex_labels, Group((1, 2), (3, 4)));;
gap> arc_labels := [[1, 2, 3], [3, 4, 5], [1, 2], [1, 2], [3,4], [4,5], [6,7]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> LocalActionDiagramIsDiscrete(lad);
false
gap> LocalActionDiagramIsUniscalar(lad);
false
gap> LocalActionDiagramIsUnimodular(lad);
true

```

2.2.21 LocalActionDiagramIsBurgerMozesUniversalGroup

▷ `LocalActionDiagramIsBurgerMozesUniversalGroup(lad)` (property)

Returns: true if the corresponding universal group is a Burger–Mozes Universal Group and false otherwise.

If a local action diagram has a single vertex and all loops at the vertex are self-reverse then the diagram corresponds to a Burger–Mozes group (see PROBABLY SECOND COLIN SIMON PAPER). If the given diagram has such a structure then this function returns true. It also sets `LocalActionDiagramGroupName` (2.2.14) to `"U({vertex_label})"` where `{vertex_label}` is the name of the group labelling the vertex.

Example

```

gap> graph := RSGraphByAdjacencyList([[1, 1], [1, 1]], ());;
gap> vertex_labels := [Group((1, 2), (3, 4))];;
gap> SetName(vertex_labels[1], "C2 x C2");
gap> arc_labels := [[1, 2], [3, 4]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> View(lad);
<LocalActionDiagram with 1 vertex and 2 arcs>
gap> LocalActionDiagramIsBurgerMozesUniversalGroup(lad);
true
gap> View(lad);
<U(C2 x C2) (as a Local Action Diagram)>

```

2.2.22 LocalActionDiagramIsEndStabiliser

▷ `LocalActionDiagramIsEndStabiliser(lad)` (property)

Returns: true if the corresponding universal group is the stabiliser of a single end in the full automorphism group of a regular tree and false otherwise. otherwise.

If a local action diagram has a single vertex with two mutually reverse loops and the vertex is labelled by S_{d-1} acting on $\{1, 2, \dots, d\}$ then the universal group is isomorphic to $\text{Aut}(T_d)_\omega$ where ω is an end of T_d . This function returns true if the group satisfies this structure and false otherwise. It also sets `LocalActionDiagramGroupName` (2.2.14) to `Aut(Td)_omega`.

Example

```

gap> graph := RSGraphByAdjacencyList([[1, 1], [1, 1]], (1, 2));;
gap> vertex_labels := [SymmetricGroup(2)];;
gap> SetPermGroupDomain(vertex_labels[1], [1,2,3]);;
gap> arc_labels := [[1, 2], [3]];;
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> View(lad);

```

```

<LocalActionDiagram with 1 vertex and 2 arcs>
gap> LocalActionDiagramIsEndStabiliser(lad);
true
gap> View(lad);
<Aut(T3)_omega (as a Local Action Diagram)>

```

2.2.23 LocalActionDiagramIsRegularTreeAutomorphismGroup

▷ `LocalActionDiagramIsRegularTreeAutomorphismGroup(lad)` (property)

Returns: true if the corresponding universal group is the full automorphism group of a regular tree and false otherwise.

If a local action diagram has a single vertex with one self-reverse loop and the vertex is labelled by S_d then the universal group is isomorphic to $\text{Aut}(T_d)$. This function returns true if the group satisfies this structure and false otherwise. It also sets `LocalActionDiagramGroupName` (2.2.14) to $\text{Aut}(T_d)$.

Example

```

gap> graph := RSGraphByAdjacencyList([[1, 1]], ());;
gap> vertex_labels := [SymmetricGroup(5)];;
gap> arc_labels := [[1..5]];
gap> lad := LocalActionDiagramFromData(graph, vertex_labels, arc_labels);;
gap> View(lad);
<LocalActionDiagram with 1 vertex and 1 arc>
gap> LocalActionDiagramIsRegularTreeAutomorphismGroup(lad);
true
gap> View(lad);
<Aut(T5) (as a Local Action Diagram)>

```

Chapter 3

IO Operations and Visualisation

3.1 IO Operations

3.1.1 Writing To A String

- ▷ `RSGraphToWritableString(graph)` (operation)
- ▷ `LocalActionDiagramToWritableString(lad)` (operation)

Returns: A String representation of the object.

These two functions turn an `RSGraph` or local action diagram into a string representation. The exact format of the representations are described in (APPENDIX REFERENCE). These functions are not designed to store the objects in the most space-efficient format. The formats are designed so that the objects can be read into a GAP session with very little overhead and so that the objects and known attributes can be recreated in a new GAP session.

Note that these functions do not perform any disk IO operations themselves. They return a string in the GAP session and it is up to the user to chose how disk IO operations with the string are done.

For an `RSGraph` the data that is always stored is:

- the vertex ids,
- the arc ids,
- the adjacency list, and
- the reverse map.

If the canonical labelling or automorphism group (REF FUNCTIONS) of the `RSGraph` are known then they are also stored.

For a local action diagram that data that is always stored is:

- the diagrams `RSGraph` (and all data known about it),
- the vertex labels, and
- the arc labels.

The data stored if it is known is:

- the diagrams scopos,

- the corresponding group type,
- if the corresponding group is discrete,
- if the corresponding group is uniscalar,
- if the corresponding group is unimodular, and
- the name of the corresponding group.

Example

```
gap> graph := RSGraphByAdjacencyList([[1, 1]], ());;
gap> RSGraphToWritableString(graph);
"1|1,1,1|P"

gap> lad := LocalActionDiagramFromData(graph, [Group((1,2))], [[1,2]]);;
gap> LocalActionDiagramGroupType(lad);;
gap> LocalActionDiagramToWritableString(lad);
"1|1,1,1|P/1:P2,1:1,2|1:1,2|LocalActionDiagramGroupType!General|LocalActionDiagramScopos! "
```

3.1.2 Reading From A String

- ▷ `RSGraphFromWritableString(string)` (operation)
 ▷ `LocalActionDiagramFromWritableString(string[, return_graph])` (operation)

Returns: The object(s) the string represents.

These function take the output from the `RSGraphToWritableString` and `LocalActionDiagramToWritableString` methods as input and returns the objects those strings represents. While it is possible to edit these strings outside of GAP these functions do not perform error checking and so this can potentially lead to invalid `RSGraph` or local action diagram objects. If the optional `return_graph` parameter is set to true then the `LocalActionDiagramFromWritableString` function will return a list whose first entry is the local action diagram object and second entry is the `RSGraph` for this local action diagram.

Any optional attributes known about the objects at the time of writing will be stored in the objects returned by these functions without needing to be recomputed. Note that the (REF canon and aut functions) attributes can be stored in the objects created by these functions without needing the `Digraphs` to be loaded even though computing these attributes required the `Digraphs` to be loaded.

Example

```
gap> graph_string := "1|1,1,1|P";;
gap> graph := RSGraphFromWritableString(graph_string);;
gap> Print(graph);
Vertices = { 1 }
Arcs = {
  1 = ( origin = 1, terminus = 1, inverse = 1 )
}
Reverse Map = ( )

gap> lad_string := "1|1,1,1|P/1:P2,1:1,2|1:1,2|LocalActionDiagramGroupType!General|LocalActionDiagramScopos! ";;
gap> lad := LocalActionDiagramFromWritableString(lad_string);;
gap> Print(lad);
Vertices = { 1 }
Arcs = {
```

```

    1 = ( origin = 1, terminus = 1, inverse = 1 )
  }
Reverse Map = ()
Vertex Labels = {
    1 = Group( [ (1,2) ] )
}
Arc Labels = {
    1 = [ 1, 2 ]
}
gap> Print("LocalActionDiagramScopos" in KnownAttributesOfObject(lad));
true
gap> Print("LocalActionDiagramIsDiscrete" in KnownAttributesOfObject(lad));
false

```

3.1.3 IO Pickling/Unpickling

▷ IO_Pickle([file,]graph) (operation)

▷ IO_Pickle([file,]lad) (operation)

Returns: IO_OK or the pickle string if the pickling was successful or IO_ERROR if there was an error.

▷ IO_Unpickle(file) (operation)

Returns: An RSGraph or Local Action Diagram object if successful or IO_ERROR if there was an error.

The IO package supports serialising data through the IO_Pickle function. Unlike the RSGraphToWritableString and LocalActionDiagramToWritableString functions these functions support recreating an arbitrary structure within the memory of a GAP session (assuming that the pickling functions have been defined for each object). In particular, it can easily recreate data structures storing objects.

For example, if `l` is a list of RSGraphs then you can write this list to the disk with a single call to IO_Pickle. This generality comes at the cost of performance. For this reason, if you have a large collection of RSGraph and local action diagram objects that you need writing to the disk we recommend using the RSGraphToWritableString and LocalActionDiagramToWritableString functions.

In the one argument version for IO_Pickle, the pickle string for the object is returned if the pickling was successful. In the two argument version, an IO file opened to write mode is supplied and the pickle string is written to this file.

The IO_Unpickle function accepts either the pickle string as input or an IO file opened to read mode which stores pickled objects. It recreates the objects in memory exactly.

As a note for developers, the RSGraph and local action diagram objects are pickled with the "magic values" RSGO and LADO respectively.

Example

```

gap> graph := RSGraphByAdjacencyList([[1, 1]], ());
gap> lad := LocalActionDiagramFromData(graph, [Group((1,2))], [[1,2]]);
gap> list := [graph, lad];
[ <RSGraph with 1 vertex and 1 arc>, <LocalActionDiagram with 1 vertex and 1 arc> ]
gap> file := IO_File("test.pickle", "w");
gap> IO_Pickle(file, list);
IO_OK
gap> IO_Close(file);

```

```
gap> file := IO_File("test.pickle", "r");;  
gap> IO_Unpickle(file);  
[ <RSGraph with 1 vertex and 1 arc>, <LocalActionDiagram with 1 vertex and 1 arc> ]
```

Chapter 4

Isomorphisms and Automorphisms

Chapter 5

Enumeration

Index

- IO_Pickle
 - for [an IO File and] a Local Action Diagram, 22
 - for [an IO File and] an RSGraph, 22
- IO_Unpickle
 - for an IO File or String, 22
- IsLocalActionDiagram
 - for an object, 11
- IsRSGraph
 - for an object, 3
- LocalActionDiagramArcIDs, 14
- LocalActionDiagramArcLabels, 13
- LocalActionDiagramArcs, 14
- LocalActionDiagramConstruct-
 - BipartitionPreserving, 13
- LocalActionDiagramFromBurgerMozes-
 - UniversalGroup, 12
- LocalActionDiagramFromData
 - for an RSGraph, list or record of vertex labels, and list or record of arc labels, 11
- LocalActionDiagramFromDataNC
 - for an RSGraph, list or record of vertex labels, and list or record of arc labels, 11
- LocalActionDiagramFromWritableString
 - for IsString, IsBool, 21
- LocalActionDiagramGroupName, 15
- LocalActionDiagramGroupType, 16
- LocalActionDiagramInArcs, 15
- LocalActionDiagramInNeighbours, 14
- LocalActionDiagramIsBurgerMozes-
 - UniversalGroup, 18
- LocalActionDiagramIsDiscrete, 16
- LocalActionDiagramIsEndStabiliser, 18
- LocalActionDiagramIsRegularTree-
 - AutomorphismGroup, 19
- LocalActionDiagramIsUnimodular, 17
- LocalActionDiagramIsUniscalar, 17
- LocalActionDiagramNumberArcs, 14
- LocalActionDiagramNumberVertices, 14
- LocalActionDiagramOutArcs, 14
- LocalActionDiagramOutNeighbours, 14
- LocalActionDiagramRegularTree, 15
- LocalActionDiagramReverseMap, 14
- LocalActionDiagramRSGraph, 13
- LocalActionDiagramScopos, 15
- LocalActionDiagramToWritableString
 - for IsLocalActionDiagram, 20
- LocalActionDiagramVertexLabels, 13
- LocalActionDiagramVertices, 13
- RSGraphAdjacencyMatrix, 6
- RSGraphArcIDs, 5
- RSGraphArcIterator, 8
- RSGraphArcs, 5
- RSGraphBipartition, 7
- RSGraphByAdjacencyList
 - for a list, permutation[, and list], 3
- RSGraphByAdjacencyListNC
 - for a list, permutation[, and list], 3
- RSGraphByAdjacencyMatrix
 - for a matrix, permutation[, and list], 4
- RSGraphByAdjacencyMatrixNC
 - for a matrix, permutation[, and list], 4
- RSGraphDegree, 7
- RSGraphFromWritableString
 - for IsString, 21
- RSGraphHasParallelArcs, 8
- RSGraphInArcs, 6
- RSGraphInNeighbours, 6
- RSGraphIsBipartite, 7
- RSGraphIsCycle, 7
- RSGraphNumberArcs, 5
- RSGraphNumberVertices, 5
- RSGraphOutArcs, 6
- RSGraphOutNeighbours, 6
- RSGraphReverseMap, 6
- RSGraphSpanningTree, 10

RSGraphSubgraph
 for an RSGraph and List of arcs, [8](#)
RSGraphSubgraphNC
 for an RSGraph and List of arcs, [8](#)
RSGraphToStandardForm, [9](#)
RSGraphToWritableString
 for IsRSGraph, [20](#)
RSGraphVertices, [5](#)